

Pontifícia Universidade Católica de Minas Gerais

Disciplina: Sistemas Operacionais

TRABALHO PRÁTICO 2

Projetando um Gerenciador de Memória Virtual

Simulador de tradução de endereços lógicos para físicos com TLB, tabela de páginas, paginação por demanda e substituição de páginas por LRU aproximado (Aging)

Integrantes

Vinícius Oliveira Ramos

Thiago Costa Soares

Repositório:

https://github.com/viniciusoramos/TabralhoPratico2_SO

Professor: Lucas Bragança da Silva

Este relatório descreve a arquitetura, a implementação, os testes e a análise de resultados do simulador de gerência de memória virtual desenvolvido em C, bem como a metodologia de desenvolvimento adotada pela equipe.

1. Introdução e Fundamentação Teórica

1.1 Memória virtual e paginação

A **memória virtual** é uma técnica que desacopla o espaço de endereçamento visto por um processo (endereços *lógicos*) do espaço físico real da memória principal (endereços *físicos*). Isso permite que o espaço lógico seja maior que a memória física disponível e que apenas as porções efetivamente utilizadas de um programa estejam residentes em memória.

A implementação mais comum da memória virtual é a **paginação**. O espaço lógico é dividido em blocos de tamanho fixo chamados **páginas**, e a memória física é dividida em blocos de mesmo tamanho chamados **quadros** (*frames*). Uma **tabela de páginas** mantém, para cada página, o quadro em que ela está armazenada.

Neste trabalho, o espaço de endereçamento virtual tem $2^{16} = 65.536$ bytes e a memória física tem **32.768 bytes**. Como a memória física é metade do espaço lógico, é necessário gerenciar a ocupação dos quadros e aplicar uma **política de substituição** quando não há quadros livres.

1.2 Estrutura do endereço lógico

Cada endereço lógico de 16 bits é interpretado como mostra a Figura 1:

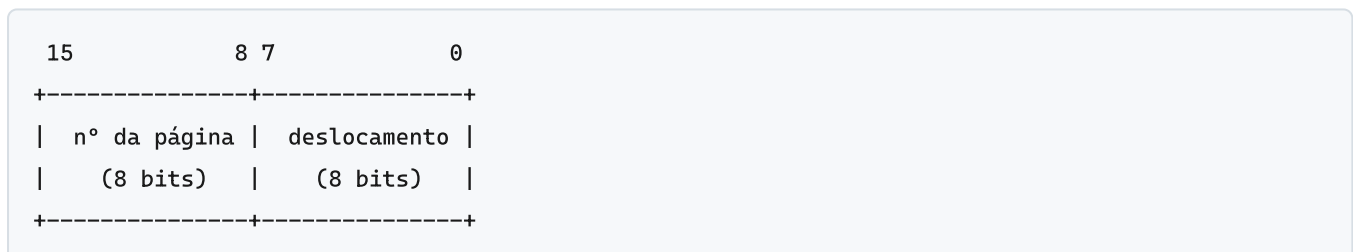


Figura 1 — Estrutura do endereço lógico (16 bits úteis).

- Os **8 bits mais significativos** (15..8) formam o **número da página** (0 a 255).
- Os **8 bits menos significativos** (7..0) formam o **deslocamento** (*offset*, 0 a 255) dentro da página.

Como o arquivo de entrada contém inteiros de 32 bits, apenas os 16 bits menos significativos são considerados, o que é feito mascarando o endereço com **0xFFFF**.

1.3 TLB (Translation Lookaside Buffer)

A consulta à tabela de páginas a cada acesso é custosa. A **TLB** é uma pequena memória associativa (16 entradas neste trabalho) que armazena as traduções **página → quadro** usadas mais recentemente. O fluxo de tradução é:

- Extrai-se o número da página do endereço lógico.
- Consulta-se a **TLB**. Se houver acerto (*TLB hit*), o quadro é obtido diretamente.
- Em caso de falha (*TLB miss*), consulta-se a **tabela de páginas**.

4. Se a página não estiver residente, ocorre uma **falta de página** (*page fault*).

1.4 Paginação por demanda e faltas de página

A **paginação por demanda** carrega uma página na memória física apenas quando ela é referenciada pela primeira vez. A "memória de retaguarda" (*backing store*) é o arquivo binário `BACKING_STORE.bin` (65.536 bytes), tratado como arquivo de **acesso aleatório**. Quando ocorre uma falta de página, lê-se uma página de 256 bytes a partir do deslocamento `página × 256` e armazena-se em um quadro livre.

1.5 Substituição de páginas: LRU aproximado por Aging

Quando não há quadros livres, escolhe-se uma página **vítima** para ser substituída. A política ideal seria o **LRU** (*Least Recently Used*), mas mantê-lo exatamente exigiria *timestamps* ou históricos completos de acesso. O **algoritmo de envelhecimento** (*Aging Algorithm*) é uma aproximação eficiente do LRU:

- Cada página residente possui um **bit de referência (R)** e um **contador de envelhecimento de 8 bits**.
- Sempre que a página é acessada, seu bit R é marcado como 1.
- Periodicamente (a cada acesso, neste trabalho), para **todas as páginas residentes**: 1. o contador é deslocado **uma posição à direita**; 2. o bit R é inserido no **bit mais significativo** do contador (`0x80`); 3. o bit R é **zerado**.
- A vítima é a página com o **menor valor do contador**.

Páginas referenciadas recentemente acumulam 1s nos bits mais significativos (contador alto); páginas há muito não usadas têm o contador "decaindo" para 0, tornando-se candidatas naturais à substituição.

2. Arquitetura e Estruturas de Dados

2.1 Organização em módulos

O projeto é organizado em módulos coesos, cada um responsável por uma parte da simulação (Tabela 1).

Tabela 1 — Módulos do projeto e suas responsabilidades.

Módulo	Arquivos	Responsabilidade
Configuração	<code>include/config.h</code>	Constantes do sistema (tamanhos, nº de quadros, caminho do <i>backing store</i>).
Programa principal	<code>src/main.c</code>	Laço de leitura/tradução de endereços e orquestração dos demais módulos.
TLB	<code>tlb.h</code> , <code>tlb.c</code>	Memória associativa com política FIFO.
Tabela de páginas	<code>page_table.h</code> , <code>page_table.c</code>	Mapeamento página→quadro, validade, e estado do LRU aproximado.
Memória física	<code>memory.h</code> , <code>memory.c</code>	Quadros, tratamento de faltas, leitura do <i>backing store</i> , seleção de vítima.
Estatísticas	<code>statistics.h</code> , <code>statistics.c</code>	Contadores e cálculo das taxas finais.

2.2 Constantes (`config.h`)

```
#define PAGE_SIZE 256          /* tamanho da página (bytes)      */
#define PAGE_TABLE_SIZE 256    /* n° de entradas da tabela      */
#define FRAME_SIZE 256         /* tamanho do quadro (bytes)     */
#define NUM_FRAMES 128         /* n° de quadros físicos         */
#define PHYSICAL_MEMORY_SIZE (FRAME_SIZE * NUM_FRAMES) /* 32.768 */
#define TLB_SIZE 16            /* n° de entradas da TLB         */
```

2.3 Principais estruturas

Entrada da TLB (`tlb.h`):

```
typedef struct {
    int page;    /* n° da página      */
    int frame;   /* n° do quadro mapeado */
    int valid;   /* 1 = entrada válida */
} tlb_entry_t;
```

A TLB é um vetor `tlb[16]`. Um índice `fifo_next` aponta a próxima posição a ser substituída, implementando a **política FIFO**.

Entrada da tabela de páginas (`page_table.h`):

```
typedef struct {
    int frame;          /* quadro associado      */
    int valid;          /* 1 = página residente */
    unsigned char reference_bit; /* bit R do LRU aproximado */
    unsigned char aging_counter; /* contador de envelhecimento */
} page_table_entry_t;
```

A tabela é um vetor `page_table[256]`.

Memória física (`memory.c`):

```
static signed char physical_memory[NUM_FRAMES][FRAME_SIZE]; /* 128 x 256 */
static int frame_to_page[NUM_FRAMES]; /* página em cada quadro (-1 livre) */
```

O vetor `frame_to_page` permite localizar rapidamente quadros livres e saber qual página ocupa cada quadro (usado também na invalidação durante a substituição).

2.4 Fluxo de tradução de um endereço

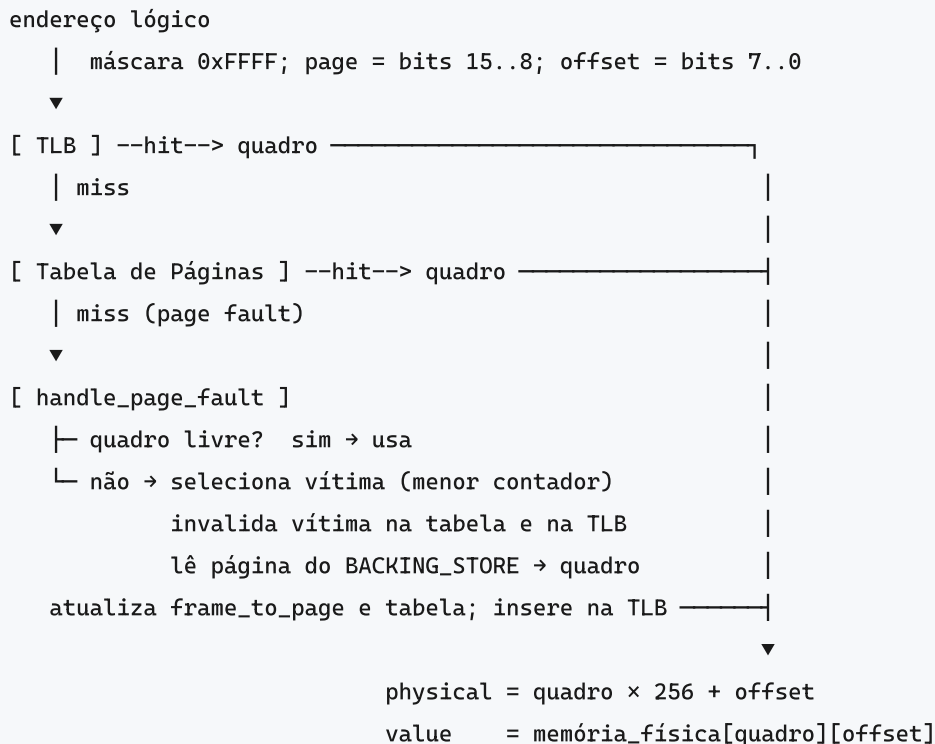


Figura 2 — Fluxo de tradução de um endereço lógico para físico.

3. Implementação e Decisões de Projeto

3.1 Extração de página e *offset* (`main.c`)

```
unsigned int masked = ((unsigned int) logical_address) & 0xFFFF;
int page    = (masked >> 8) & 0xFF;
int offset  = masked & 0xFF;
```

- A conversão para `unsigned int` antes do deslocamento evita comportamento dependente de sinal em deslocamentos à direita.
- O endereço lógico **original** é preservado para impressão (os endereços de teste já estão no intervalo válido, então o valor mascarado coincide com o original).

O **endereço físico** é a concatenação do quadro com o *offset*:

```
int physical_address = frame * FRAME_SIZE + offset;
```

3.2 TLB com política FIFO (`tlb.c`)

- `tlb_lookup`: busca linear por uma entrada válida cuja página coincida; retorna o quadro ou `-1`.
- `tlb_insert` segue a política ditada pelo *template*: 1. se a página **já está** na TLB, apenas atualiza o quadro (preserva a ordem FIFO); 2. senão, se houver **entrada inválida**, reutiliza-a; 3. senão (TLB cheia),

substitui a entrada apontada por `fifo_next` e avança `fifo_next = (fifo_next + 1) % TLB_SIZE`.

- `tlb_remove`: invalida a entrada de uma página removida da memória física, evitando que a TLB retorne um mapeamento obsoleto.

Decisão: preferir reutilizar entradas inválidas antes de aplicar o FIFO mantém a TLB sempre cheia o quanto possível e respeita a especificação do *template*.

3.3 Tabela de páginas (`page_table.c`)

- `page_table_lookup`: retorna o quadro se a página é válida; senão `-1` (sinaliza falta).
- `page_table_update`: registra `página→quadro`, marca como válida e **reinicia** o contador de envelhecimento e o bit R (página recém-carregada começa "do zero").
- `page_table_invalidate`: zera a entrada quando a página é removida (substituição).
- `page_table_set_reference`: marca o bit R da página acessada.
- `page_table_update_aging`: aplica o algoritmo de envelhecimento a todas as páginas válidas: `c`
`aging_counter >= 1; if (reference_bit) aging_counter |= 0x80; reference_bit = 0;`

3.4 Tratamento de faltas de página (`memory.c`)

`handle_page_fault` implementa a paginação por demanda: 1. procura um quadro livre (`find_free_frame`); 2. se não houver, seleciona a vítima (`select_victim_page`), **invalida** sua entrada na tabela de páginas e a **remove da TLB**, liberando seu quadro; 3. posiciona o arquivo com `fseek(backing, (long) page * PAGE_SIZE, SEEK_SET)` e lê `PAGE_SIZE` bytes com `fread` para `physical_memory[frame]`; 4. atualiza `frame_to_page[frame]` e a tabela de páginas; 5. retorna o número do quadro.

O retorno de `fread` / `fseek` é verificado para detectar I/O incompleto.

`select_victim_page` percorre as páginas válidas e escolhe a de **menor contador de envelhecimento**; em empate, a de **menor número de página** (critério consistente, conforme permitido pela especificação).

3.5 Ordem de atualização do LRU (`main.c`)

A cada acesso, **após** resolver a tradução, executa-se:

```
page_table_set_reference(page); /* marca R da página acessada */
page_table_update_aging();      /* envelhece todas as válidas */
```

Assim, a página recém-acessada termina o ciclo com o bit mais significativo do contador em 1 (valor alto ⇒ "recentemente usada"), enquanto as demais decaem. Essa é a interpretação do intervalo de atualização "a cada acesso à memória" sugerida pela especificação.

3.6 Leitura do valor e estatísticas

- `read_memory(frame, offset)` retorna `physical_memory[frame][offset]` como `signed char` (valores > 127 aparecem negativos, conforme esperado).
- As taxas finais são frações sobre o total de acessos, com proteção contra divisão por zero: `c`
`page_fault_rate = (double) page_faults / total_addresses; tlb_hit_rate = (double) tlb_hits / total_addresses;`

3.7 Dificuldades encontradas

Durante o desenvolvimento, alguns pontos exigiram análise cuidadosa e algumas iterações até a solução final:

- **Interpretação do intervalo de atualização do Aging.** A especificação descreve a atualização dos contadores como periódica, citando "a cada acesso à memória" apenas como exemplo. Foi preciso decidir a granularidade do envelhecimento; optou-se por envelhecer todas as páginas válidas a cada acesso, o que reproduz fielmente o comportamento de LRU aproximado esperado.
- **Ordem entre marcar a referência e envelhecer.** Definir se `page_table_set_reference` deveria ser chamado antes ou depois de `page_table_update_aging` não era óbvio. Colocando a marcação **antes**, a página recém-acessada encerra o ciclo com o bit mais significativo do contador em 1 (valor alto), o que corresponde corretamente a "recentemente usada".
- **Tratamento do byte sinalizado.** Como `char` em C pode ser sinalizado, valores acima de 127 aparecem negativos na saída (`Value`). Foi necessário usar `signed char` de forma consistente e, na extração do endereço, **converter para unsigned antes do deslocamento à direita**, evitando comportamento dependente de sinal.
- **Consistência entre TLB e tabela de páginas na substituição.** Ao remover uma página vítima, era fácil esquecer de invalidar a entrada correspondente no TLB, o que deixaria um **mapeamento obsoleto** — a TLB apontaria para um quadro que passou a conter outra página. Garantir a invalidação dupla (tabela e TLB) foi um ponto de atenção recorrente.
- **I/O de acesso aleatório no BACKING_STORE.bin.** Foi preciso calcular corretamente o deslocamento (`página × 256`), posicionar o arquivo com `fseek` e validar o retorno de `fread` para detectar leituras incompletas.
- **Validação sem arquivos de referência.** Sem dispor das saídas de referência durante o desenvolvimento, garantir a correção foi um desafio. A solução foi conceber o **invariante determinístico** e o **simulador de referência independente** descritos na Seção 4.1, que deram confiança total no resultado.

4. Testes Realizados e Análise dos Resultados

4.1 Metodologia de validação

A correção foi verificada por **três métodos independentes e complementares**:

(a) Invariante de verdade absoluta (ground truth). O `BACKING_STORE.bin` gerado por `generate_data.py` armazena, em cada posição `p`, o byte `p % 256`. Como o byte lido fica na posição `página × 256 + offset = endereço`, segue que, para **qualquer** endereço, uma tradução correta deve produzir:

```
Value == (signed char)(endereço & 0xFF)
```

Esse invariante independe de quadros, TLB ou substituição — é determinado apenas pela correção da tradução e da leitura. Verificou-se que ele vale para **todos os 10.000 endereços** de **ambos** os arquivos de teste (0 divergências).

(b) Reimplementação de referência independente. Implementou-se, **a partir da especificação** (e não do código C), um simulador de referência em Python com as mesmas políticas (FIFO na TLB, *Aging* na substituição, paginação por demanda). A saída do programa C foi comparada **linha a linha** com a do

simulador: as **10.005 linhas** de saída (10.000 traduções + 5 linhas de estatística) coincidem **integralmente** para os dois arquivos de teste.

(c) Compilação sem avisos. O projeto compila com `-Wall -Wextra -O2 -std=c11` sem nenhum aviso ou erro.

4.2 Resultados

Arquivo de endereços aleatórios (`addresses_random.txt`):

```
Number of Translated Addresses = 10000
Page Faults = 4958
Page Fault Rate = 0.496
TLB Hits = 654
TLB Hit Rate = 0.065
```

Arquivo com localidade de referência (`addresses_location.txt`):

```
Number of Translated Addresses = 10000
Page Faults = 1164
Page Fault Rate = 0.116
TLB Hits = 7925
TLB Hit Rate = 0.792
```

Quadro comparativo (Tabela 2):

Tabela 2 — Comparativo de resultados entre os dois arquivos de teste.

Métrica	Aleatório	Localidade
Endereços traduzidos	10.000	10.000
Faltas de página	4.958	1.164
Taxa de faltas de página	0,496	0,116
Acertos da TLB	654	7.925
Taxa de acerto da TLB	0,065	0,792
Páginas distintas acessadas	256	256
Faixa de endereços físicos	[0, 32767]	[19, 32723]

4.3 Análise

- Localidade de referência é decisiva.** Embora ambos os arquivos acessem todas as 256 páginas, o arquivo com localidade concentra acessos em páginas próximas. Isso eleva a **taxa de acerto da TLB de 6,5% para 79,2%** e reduz a **taxa de faltas de página de 49,6% para 11,6%** — exatamente o comportamento previsto pela teoria de localidade.
- Pressão de capacidade no caso aleatório.** Existem 256 páginas e apenas 128 quadros. No acesso aleatório, qualquer página tem ~50% de chance de não estar residente, o que explica a taxa de faltas

próxima de 0,5. Das 4.958 faltas, 256 são **faltas obrigatórias** (primeiro acesso de cada página) e as demais (~4.702) são **faltas de capacidade** causadas pela substituição.

- **TLB pequena × acesso disperso.** Com apenas 16 entradas e acesso espalhado por 256 páginas, a TLB raramente retém a tradução necessária no caso aleatório (6,5%). Sob localidade, o conjunto de trabalho cabe na TLB com frequência, daí os 79,2% de acerto.
- **Uso da memória física.** No caso aleatório, os endereços físicos cobrem toda a faixa `[0, 32767]`, confirmando que os 128 quadros são efetivamente utilizados e reciclados pela política de substituição.
- **Eficácia do Aging.** A política aproximada de LRU mantém residentes as páginas com acessos recentes; sob localidade, isso resulta em poucas faltas mesmo com memória física igual à metade do espaço lógico.

5. Conclusão

O simulador implementa corretamente todas as etapas da tradução de endereços lógicos para físicos: extração de página/offset, consulta à **TLB** (com substituição **FIFO**), consulta e atualização da **tabela de páginas, paginação por demanda** a partir do `BACKING_STORE.bin`, **substituição de páginas por LRU aproximado** (*Aging*) com invalidação consistente na tabela e na TLB, e o cálculo das **estatísticas** finais.

A correção foi comprovada por um invariante determinístico verificado em 100% dos casos e pela equivalência linha a linha com uma reimplementação independente da especificação. Os resultados quantitativos reproduzem o comportamento esperado pela teoria: a localidade de referência reduz drasticamente as faltas de página e eleva a taxa de acerto da TLB. O trabalho consolidou, na prática, os conceitos de memória virtual, paginação, TLB e políticas de substituição.

6. Uso de Inteligência Artificial (IA)

6.1 Visão geral e divisão de papéis

A condução do projeto — concepção da arquitetura, escolha dos algoritmos e políticas, definição das especificações, revisão crítica do código e validação dos resultados — foi realizada **pela equipe**. A IA foi empregada como **ferramenta de apoio**, operada sob a direção do grupo: a partir das especificações elaboradas pelos integrantes, ela auxiliou na **geração de trechos de código** das funções marcadas com `TODO` e no **rascunho deste relatório**. Todo o material produzido foi **revisado, testado e ajustado** pela equipe, que é **integralmente responsável** pelo código entregue e capaz de explicar e defender cada decisão.

- **Ferramenta:** Claude (Anthropic), usada como assistente de programação.
- **O que a IA fez:** sugeriu implementações para as funções a partir das especificações da equipe e apoiou a redação do relatório.
- **O que a equipe fez:** ver a Seção 6.2.

6.2 Contribuições da equipe

As decisões de engenharia que definem o comportamento do simulador foram tomadas pelos integrantes, entre elas:

- **Arquitetura e modelagem de dados** — separação em módulos (TLB, tabela de páginas, memória física, estatísticas) e definição das estruturas (`tlb_entry_t` , `page_table_entry_t` , `frame_to_page`).
- **Escolha das políticas** — FIFO no TLB (com reuso de entradas inválidas antes da substituição), *Aging* de 8 bits para o LRU aproximado, e o critério de desempate na seleção da vítima (menor número de página).
- **Decisões de implementação** — interpretação do intervalo de atualização do *Aging* ("a cada acesso"), ordem entre marcar referência e envelhecer os contadores, reinício do contador em páginas recém-carregadas e tratamento de I/O do `BACKING_STORE.bin` .
- **Revisão crítica** — análise do código gerado e endurecimento defensivo (por exemplo, a verificação de limites na seleção de vítima em `handle_page_fault` , mantendo a consistência com o restante do projeto).
- **Metodologia de validação** — concepção do **invariante determinístico** (`Value == (signed char) (endereço & 0xFF)`) e da **reimplementação de referência** para comparação linha a linha (ver Seção 4.1), que demonstram o entendimento aprofundado do problema.

6.3 Metodologia — *Spec-Driven Development* (SDD)

O trabalho com a IA seguiu o modelo **dirigido por especificação**: em vez de pedidos vagos, a equipe redigiu, para cada módulo, uma especificação clara contendo o problema, os requisitos funcionais/não funcionais e as restrições de implementação (linguagem, interfaces e políticas). A ordem seguiu a sugestão do enunciado (extração de endereço → *backing store* → tabela de páginas → memória física → faltas → substituição → TLB → estatísticas). As especificações abaixo — base dos prompts utilizados — são, portanto, **contribuição intelectual do grupo**.

6.4 Especificações utilizadas (prompts)

Especificação 1 — Tradução de endereços (`main.c`):

"Implemente a extração de página e *offset* de um endereço lógico de 16 bits (mascarar com `0xFFFF` ; 8 bits de página em 15..8 e 8 bits de *offset* em 7..0) e o cálculo do endereço físico (`frame × FRAME_SIZE + offset`). Restrições: C11, preservar o endereço lógico original para impressão; usar aritmética sem sinal no deslocamento."

Especificação 2 — TLB com FIFO (`tlb.c`):

"Implemente `tlb_lookup` , `tlb_insert` e `tlb_remove` para uma TLB de 16 entradas. Política de inserção: (1) se a página já existe, atualizar o quadro; (2) usar a primeira entrada inválida; (3) se cheia, substituir por FIFO usando o índice `fifo_next` . `tlb_remove` deve invalidar a entrada de uma página removida da memória física."

Especificação 3 — Tabela de páginas e LRU aproximado (`page_table.c`):

"Implemente consulta, atualização e invalidação da tabela de páginas (256 entradas). Implemente o algoritmo de *Aging* de 8 bits: a cada atualização, deslocar o contador à direita, inserir o bit de referência no bit mais significativo (`0x80`) e zerar o bit de referência. Páginas recém-carregadas devem reiniciar contador e bit de referência."

Especificação 4 — Faltas de página e substituição (`memory.c`):

"Implemente `handle_page_fault` com paginação por demanda: procurar quadro livre; se não houver, selecionar a vítima de **menor contador de envelhecimento** (empate: menor número de página), invalidar a vítima na tabela e na TLB e reutilizar seu quadro; ler 256 bytes do `BACKING_STORE.bin` na posição `página × 256` (via `fseek / fread`, com verificação de I/O); atualizar `frame_to_page` e a tabela. Implemente também `read_memory` e `select_victim_page`."

Especificação 5 — Estatísticas (`statistics.c`):

"Implemente os contadores de endereços, faltas de página e acertos da TLB, e o cálculo das taxas (`faltas/total` e `acertos/total`) com proteção contra divisão por zero, mantendo o formato de saída existente."

Especificação 6 — Validação (testes), concebida pela equipe:

"Crie um método de validação independente: (1) verificar o invariante `Value == (signed char) (endereço & 0xFF)` decorrente do `BACKING_STORE` determinístico; (2) reimplementar o simulador a partir da especificação em outra linguagem e comparar a saída linha a linha com o programa C para os dois arquivos de teste."

6.5 Responsabilidade técnica

O grupo compreende e é capaz de explicar todas as estruturas de dados, algoritmos e decisões implementadas. A IA atuou como ferramenta de apoio à codificação e à redação, **sob a direção e revisão da equipe**, sem substituir o entendimento do projeto. A responsabilidade técnica pelo código entregue é integralmente dos integrantes.

7. Repositório e Instruções de Execução

Repositório público (GitHub): https://github.com/viniciusoramos/TabralhoPratico2_SO

O repositório mantém a estrutura do projeto-base, com todas as implementações realizadas e este relatório (em `report/`).

```
# 1) Gerar os arquivos de entrada
cd data
python3 generate_data.py
cd ..

# 2) Compilar
make                # gera o executável ./vm

# 3) Executar
./vm < data/addresses_random.txt
./vm < data/addresses_location.txt
```

Em ambientes sem `make` / `gcc`, o projeto também compila com qualquer compilador C11, por exemplo: `zig cc -Wall -Wextra -O2 -std=c11 -Iinclude src/*.c -o vm`.

8. Referências

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. — capítulos de Memória Virtual e Paginação por Demanda.
- TANENBAUM, A. S. *Modern Operating Systems* — algoritmo de envelhecimento (*Aging*) como aproximação do LRU.